Circuit Breaker Integration — order-easy

What is Circuit Breaker?

Imagine calling someone on the phone. They don't pick up. You try again. And again. You're stuck wasting time in a loop.

Circuit Breaker says: "Stop trying after a certain number of failures. Give an instant fallback response. Try again after some time."

In our project, `order-service` calls `inventory-service` via Feign (synchronous HTTP). If inventory-service is down and there is no circuit breaker, order-service will keep trying, timeout after 5 seconds every time, and the user waits forever — or worse, the system crashes under load.

Circuit Breaker makes it **fail-fast** — instant fallback response instead of hanging.

Why is it needed in order-easy?

Without Circuit Breaker	With Circuit Breaker
Feign call hangs for 5+ seconds	Fails in milliseconds
Thread pool gets exhausted	Thread freed immediately
Other requests also start failing	Only affected service is isolated
User sees timeout / 500 error	User sees clean "service unavailable" message
System can cascade crash	System stays stable

The 3 States

```
CLOSED  →  (5 out of 10 calls fail)  →  OPEN
OPEN    →  (wait 10 seconds)         →  HALF-OPEN
HALF-OPEN → (3 test calls pass)      →  CLOSED
HALF-OPEN → (any test call fails)    →  OPEN
```

- **CLOSED** — Normal mode. All requests pass through. CB just watches.
- **OPEN** — Failure threshold crossed. All requests blocked instantly. Fallback returned.
- **HALF-OPEN** — After wait time, allows a few test requests to check if service recovered.

Why Feign only — NOT Kafka?

Feign is synchronous. Order-service WAITS for inventory-service to respond. If it's down, the thread hangs. Circuit breaker saves us.

Kafka is asynchronous. Order-service publishes an event and moves on immediately — it doesn't wait. If delivery-service is down, the Kafka event stays in the topic and gets consumed when the service comes back. Kafka handles resilience itself.

> Rule: **Circuit Breaker only on Feign calls. Never on Kafka.**

Integration Steps

Step 1 — Add Dependencies (pom.xml)

Both `order-service` and `cart-service`:

```xml
<!-- Resilience4j — the circuit breaker library -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>

<!-- AOP — needed to intercept @CircuitBreaker annotated methods -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Why AOP? The `@CircuitBreaker` annotation works because AOP wraps the method. When the method is called, AOP intercepts it first, checks the circuit state, and decides whether to allow the call or trigger the fallback. Without AOP, the annotation does nothing.

Step 2 — Add Feign Timeouts (application.properties)

```properties
feign.client.config.default.connectTimeout=3000
feign.client.config.default.readTimeout=5000
```

- `connectTimeout=3000` — If we can't reach the server in 3 seconds → treat as failure

- `readTimeout=5000` — If server doesn't respond within 5 seconds → treat as failure

Why? Without timeouts, a hanging server makes Feign wait forever. Timeouts ensure failures are detected quickly so the circuit breaker can react.

Step 3 — Configure CB Instances (application.properties)

order-service:

```properties
# Circuit breaker for inventory-service calls
resilience4j.circuitbreaker.instances.inventoryServiceCB.failureRateThreshold=5
resilience4j.circuitbreaker.instances.inventoryServiceCB.waitDurationInOpenStat
resilience4j.circuitbreaker.instances.inventoryServiceCB.slidingWindowSize=10
resilience4j.circuitbreaker.instances.inventoryServiceCB.permittedNumberOfCalls

# Circuit breaker for cart-service calls
resilience4j.circuitbreaker.instances.cartServiceCB.failureRateThreshold=50
resilience4j.circuitbreaker.instances.cartServiceCB.waitDurationInOpenState=10s
resilience4j.circuitbreaker.instances.cartServiceCB.slidingWindowSize=10
resilience4j.circuitbreaker.instances.cartServiceCB.permittedNumberOfCallsInHal

# Actuator — expose circuit breaker state
management.endpoints.web.exposure.include=health,info,metrics
# Check state at: /actuator/metrics/resilience4j.circuitbreaker.calls
```

cart-service:

```properties
```

```
# Circuit breaker for product-service calls
resilience4j.circuitbreaker.instances.productServiceCB.failureRateThreshold=50
resilience4j.circuitbreaker.instances.productServiceCB.waitDurationInOpenState=
resilience4j.circuitbreaker.instances.productServiceCB.slidingWindowSize=10
resilience4j.circuitbreaker.instances.productServiceCB.permittedNumberOfCallsIn
```

What each setting means:

Setting	Value	Meaning
`failureRateThreshold`	50%	If 5 out of 10 calls fail → open the circuit
`waitDurationInOpenState`	10s	Stay OPEN for 10 seconds before trying HALF-OPEN
`slidingWindowSize`	10	Watch the last 10 calls to calculate failure rate
`permittedCalls HalfOpen`	3	Allow 3 test calls in HALF-OPEN state

Step 4 — Create ExternalServiceProxy.java (order-service)

```
java
```

```java
@Service
public class ExternalServiceProxy {

    @Autowired
    private InventoryClient inventoryClient;

    @Autowired
    private CartClient cartClient;

    @CircuitBreaker(name = "inventoryServiceCB", fallbackMethod = "handleInvent
    public boolean checkInventory(Long productId) {
        return inventoryClient.checkStock(productId);
    }

    public boolean handleInventoryFallback(Long productId, Throwable t) {
        log.warn("fail-fast with graceful degradation triggered. Error: {}", t.
        throw new ServiceUnavailableException("Inventory service unavailable. P
    }

    @CircuitBreaker(name = "cartServiceCB", fallbackMethod = "handleCartFallbac
    public CartResponse getCart(Long userId) {
        return cartClient.getCart(userId);
    }

    public CartResponse handleCartFallback(Long userId, Throwable t) {
        log.warn("fail-fast with graceful degradation triggered. Error: {}", t.
        throw new ServiceUnavailableException("Cart service unavailable. Please
    }
}
```

Why a Proxy class? Spring AOP cannot intercept a method that calls another method in the same class (self-call). If `@CircuitBreaker` is placed directly in `OrderService` on a method

that calls its own Feign client, AOP won't intercept it. The separate `ExternalServiceProxy` bean solves this — AOP intercepts cross-bean calls correctly.

Step 5 — Update OrderService.java

```java
@Service
public class OrderService {

    @Autowired
    private ExternalServiceProxy proxy;  // NOT InventoryClient directly

    public OrderResponse createOrder(CreateOrderRequest req) {
        // CB logic happens inside proxy — AOP intercepts the proxy call
        boolean inStock = proxy.checkInventory(req.getProductId());

        if (!inStock) {
            throw new OutOfStockException("Product out of stock");
        }

        // Save order, publish Kafka event
        Order order = orderRepository.save(buildOrder(req));
        kafkaTemplate.send("order-placed", new OrderEvent(order));

        return new OrderResponse(order.getId(), "CONFIRMED");
    }
}
```

Step 6 — Add GlobalExceptionHandler.java

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ServiceUnavailableException.class)
    @ResponseStatus(HttpStatus.SERVICE_UNAVAILABLE)
    public ApiResponse handleServiceDown(ServiceUnavailableException e) {
        return ApiResponse.builder()
            .status("FAILED")
            .message(e.getMessage())
            .data(null)
            .timestamp(LocalDateTime.now())
            .build();
    }
}
```

Response sent to client when circuit is OPEN:

```json
{
  "status": "FAILED",
  "message": "Inventory service unavailable. Please try again.",
  "data": null,
  "timestamp": "2026-04-21T10:30:00"
}
```

Complete Runtime Flow

```
Client
  ↓ POST /api/orders/place
API Gateway (port 8080)
  ↓ lb://order-service → Eureka → 192.168.1.10:8083
OrderService.createOrder()
  ↓ calls proxy.checkInventory()
AOP intercepts the method call
  ↓ checks circuit state

  ┌─ CLOSED? → Feign call goes to inventory-service
  │               ↓ response returned
  │           order saved → Kafka event published → 201 OK
  │
  └─ OPEN?   → Feign call BLOCKED instantly
               fallback triggered
               503 JSON returned to client in milliseconds

After 10s → HALF-OPEN → 3 test calls
  ├─ all pass? → CLOSED (normal traffic resumes)
  └─ any fail? → OPEN again (wait another 10s)
```

How to Verify

```bash
```

```
# 1. Check circuit breaker state
GET http://localhost:8083/actuator/health

# 2. Stop inventory-service, then place an order
# → Should get 503 JSON instantly (not a timeout)

# 3. Start inventory-service again
# → After 10s, CB enters HALF-OPEN
# → Place order again → CB closes → normal

# 4. Check metrics
GET http://localhost:8083/actuator/metrics/resilience4j.circuitbreaker.calls
```

Interview One-Liner

> *"I added Resilience4j Circuit Breakers only on Feign calls — inventory and cart in order-service, product in cart-service. Kafka flows are intentionally excluded because Kafka is async and handles retries itself. The key design decision was using an* `ExternalServiceProxy` *pattern to avoid Spring AOP self-invocation issues. When a downstream service fails, the circuit opens after 50% failure rate over 10 calls, and we return a structured 503 JSON response immediately — fail-fast instead of hanging."*